

Otras consideraciones importantes

Por: Ricardo Cuellar



Punteros para PATCH

Al usar valores normales no se puede distinguir si no se envió un campo o si se envió un valor vacío. Con punteros, nil significa no enviado y podemos saber si un campo está vacío solamente.

```
updateTaskRequest.go

type UpdateTaskRequest struct {
    Title      *string `json:"title,omitempty"`
    Description *string `json:"description,omitempty"`
    Done       *bool   `json:"done,omitempty"`
}
```



DisallowUnknownFields

Es una práctica de seguridad. Ya que si el cliente envía un campo que no existe en nuestro struct, Go lo ignora de forma silenciosa. Con esto el Decoder devuelve error si hay campos desconocidos, nos protege de campos no deseados o inyectados.

```
disallowUnknownFields.go

decoder := json.NewDecoder(r.Body)
decoder.DisallowUnknownFields()

var req CreateTaskRequest
if err := decoder.Decode(&req); err != nil {
    http.Error(w, "invalid request body", http.StatusBadRequest)
    return
}
```



DisallowUnknownFields

```
disallowUnknownFields.go

type CreateTaskRequest struct {
    Title      string `json:"title"`
    Description string `json:"description"`
}
```

```
valid.json

{
  "title": "Aprender Go",
  "description": "Practicar JSON"
}
```

Válido

```
valid.json

{
  "title": "Aprender Go",
  "description": "Practicar JSON",
  "isAdmin": true
}
```

No válido



RawMessage

Es útil cuando la forma del JSON depende de otro campo del mismo JSON.

```
example.go|  
  
type WebhookEvent struct {  
    Type      string      `json:"type"`  
    Payload   json.RawMessage `json:"payload"`  
}
```



RawMessage

```
example.json
{
  "type": "task.created",
  "payload": {
    "id": 1,
    "title": "Aprender Go"
  }
}
```

```
example.json
{
  "type": "task.deleted",
  "payload": {
    "id": 1
  }
}
```



Gracias



www.devtalles.com



[/ricardocuellars](https://www.linkedin.com/company/ricardocuellars)



[@ricardocuellars](https://twitter.com/ricardocuellars)

